The Journal of Open Source Software

DMRG++: DMRG calculations across time, frequency, and temperature

Gonzalo Alvarez [1¶], **Daniel Arndt** [1], **Peter Doak** [1], **Steven Hahn** [1], **Damien Lebrun-Grandie** [1,2], **Thomas A. Maier** [1], and **Andrey Prokopenko** [1]

1 Oak Ridge National Laboratory **2** Los Alamos National Laboratory ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 12 August 2026
Published: unpublished

Summary

DMRG++ is an open-source C++ application for simulations of strongly correlated quantum systems using the density matrix renormalization group (DMRG) (White, 1992, 1993). The DMRG algorithm is used to study many-body Hamiltonians in condensed-matter physics, quantum chemistry, and Hamiltonian lattice gauge theory. DMRG++ focuses on condensed-matter models and provides an input-driven framework for selecting the physical model, geometry, interactions, symmetries, and numerical method.

DMRG++ computes ground states and correlation functions as well as frequency-dependent, time-dependent, and finite-temperature quantities. It supports built-in geometry kinds together with input-defined connections and coupling values. Researchers can therefore study different strongly correlated systems without writing a separate DMRG implementation for each Hamiltonian.

Scientific background

Many problems in condensed-matter physics, quantum chemistry, and Hamiltonian lattice gauge theory can be expressed as an eigenvalue problem,

$$H|\psi\rangle = E|\psi\rangle,$$

where the Hamiltonian H describes interacting local degrees of freedom. These degrees of freedom may be lattice sites, electronic orbitals, or lattice-gauge links and sites. If each of L local degrees of freedom has d states, the dimension of the full Hilbert space is

$$\dim \mathcal{H} = d^L.$$

Fixing particle number or other conserved quantities reduces this space, but its dimension still grows combinatorially. Directly storing an arbitrary state or diagonalizing the full Hamiltonian therefore becomes impractical after few sites.

Numerical methods avoid this cost by sampling or restricting the many-body space. Auxiliary-field quantum Monte Carlo, for example, can have polynomial cost when its sign or phase problem is absent, but polynomial scaling does not hold generally (Motta & Zhang, 2018). DMRG instead uses the restricted entanglement of many low-energy states. Ground states of local, gapped one-dimensional Hamiltonians satisfy an entanglement area law and can be approximated efficiently by matrix product states (Hastings, 2007; Schollwöck, 2011).

In the conventional formulation used by DMRG++, the lattice is divided into system and environment blocks. At each step, the reduced density matrix ranks the block states by their contribution to the target state. DMRG retains the states with the largest density-matrix eigenvalues and discards the rest, so an exponentially large Hilbert space need not be represented explicitly (White, 1992). Repeated sweeps across the lattice optimize the retained basis. The sum of the discarded eigenvalues and convergence with the number of retained states provide practical diagnostics of truncation error. In the equivalent modern description, DMRG variationally optimizes a matrix product state (MPS), and the number of retained states is its bond dimension (Schollwöck, 2011).

DMRG is most effective for chains, where a moderate bond dimension often gives high accuracy and the cost at fixed bond dimension is polynomial in the chain length. Ladders can be mapped to a one-dimensional ordering, but the entanglement and required bond dimension generally grow exponentially with the ladder width, that is, its short or transverse dimension (Stoudenmire & White, 2012). DMRG++ supplies the model, geometry, symmetry, measurement, and solver infrastructure needed to apply this procedure to a range of interacting Hamiltonians.

Statement of need

Research applications of DMRG often require changes to the Hamiltonian, lattice connectivity, conserved quantities, target states, and observables. Implementing these changes in a problem-specific code can duplicate work on basis construction, sweeps, measurements, and validation. Researchers studying strongly correlated lattice Hamiltonians therefore need reusable solver infrastructure that can accommodate different physical models and calculation protocols without requiring a new DMRG implementation for each study.

DMRG++ (Gonzalo Alvarez, 2009) addresses this need as an input-driven C++ application implementing conventional DMRG (White, 1992). Users select from model implementations compiled into the application and specify interaction-dependent connections and coupling matrices in the input. Representative models include one-, two-, and three-band Hubbard variants, Kondo models, Heisenberg and Kitaev spin models, and Holstein and Hubbard-Holstein electron-phonon models. Built-in geometry kinds and general connection matrices or coupling values supplied through the input share the same solver machinery, and calculations can use the conserved quantum-number sectors supported by each model.

The shared infrastructure supports ground-state and static-correlation calculations, time evolution, frequency-dependent response, and finite-temperature observables (G. Alvarez et al., 2011; G. Alvarez, 2013; Nocera & Alvarez, 2016a, 2016b). Researchers can therefore change a Hamiltonian or calculation protocol while retaining the same simulation and analysis workflow.

State of the field

The broad applicability of DMRG has produced a diverse software ecosystem. A recent survey identified more than 50 implementations and compared 37 packages in detail (Sehlstedt et al., 2026). These span conventional DMRG and MPS or tensor-network formulations, application-oriented programs and general-purpose libraries, and several scientific domains. Their differing approaches to Hamiltonian construction, symmetries, targeting, and numerical backends reflect distinct research workflows rather than a single hierarchy of implementations.

Independent packages nevertheless reproduce substantial core functionality. Sehlstedt et al. found overlap in symmetry handling and high-performance strategies, while most packages provide their own Lanczos- or Davidson-based eigensolver (Sehlstedt et al., 2026). Such specialization serves individual domains but increases the work needed to maintain numerical infrastructure and adopt new backends. Shared numerical components, standard interfaces, and collaboration offer complementary ways to reduce that duplication.

A useful contrast is the Julia implementation of ITensor, which provides a general-purpose library for constructing tensor-network calculations (Fishman et al., 2022). In the current Julia package organization, `ITensors.jl` provides indexed-tensor abstractions, while `ITensorMPS.jl` provides finite MPS and MPO algorithms, including DMRG.[1] Users generally write or adapt Julia code to define sites and operators, construct an MPO for the Hamiltonian, prepare an MPS, and invoke library algorithms. DMRG++, by contrast, is a C++20 application implementing conventional DMRG. Its models, targeting methods, and solver components are compiled into the application, while users select supported calculations and specify their parameters through input files. The contrast is therefore between reusable tensor abstractions for constructing applications and an integrated, domain-focused application, not a ranking of their capabilities.

The widespread use and greater generality of MPS and MPO interfaces do not make conventional DMRG obsolete. Block-based DMRG and variational MPS optimization are closely related formulations of the same method (Schollwöck, 2011). The MPS/MPO language exposes tensor-network structure directly and supports reuse across a broad range of algorithms. A mature conventional implementation can instead organize model-specific operators, symmetry sectors, targeting methods, measurements, checkpointing, and post-processing around a shared solver. This allows DMRG++ users to run supported workflows without constructing and maintaining a separate tensor-network application, while preserving validated input, regression-test, and analysis workflows. Which organization is more useful therefore depends in part on whether a user needs general tensor-network building blocks or an established application for a supported class of calculations.

DMRG++ adopts the application-oriented side of this distinction as a free and open-source C++ implementation of conventional DMRG. Development began at Oak Ridge National Laboratory in 2008 to solve condensed-matter problems with systematically monitored truncation error; the first software paper appeared in 2009 (Gonzalo Alvarez, 2009). It uses generic programming and a comparatively small software dependency set (Sehlstedt et al., 2026). Accelerator-oriented batched kernels were originally developed in the separate `DmrgppPluginSc` repository and were integrated into the main DMRG++ repository in 2026.[2]

Software design

DMRG++ is an input-driven C++20 scientific application that combines generic programming with runtime configuration. The principal `dmrg` executable reads and validates a single input file, after which `DmrgRunner` assembles the types and objects needed for the calculation. In addition to setting runtime parameters such as specifying real or complex arithmetic, choosing a Hamiltonian matrix-vector implementation, the physical model, and the targeting method, the input language has grown over time into a flexible mini-language for defining computations at runtime. It has a defined syntax for scoped labels, vector and matrix values, arithmetic expressions, and runtime substitutions. In the targeting and measurement layers syntax making use of Dirac bra and ket notaion can be used to accomplish considerable runtime composition.

The calculation has four main layers: geometry, model, DMRG engine, and targeting and measurement.

The model and geometry layers separate the local physics from the connections between sites. A model defines its local Hilbert space, operators, Hamiltonian terms, parameters, and conserved quantum numbers. The geometry defines the site ordering, connections, and coupling values for each interaction term. Thus, the same model can use different built-in geometry kinds or general connection matrices and coupling values supplied through the input, while the geometry machinery is shared by different models. Geometry implementations themselves

[1] See the official documentation for `ITensors.jl` and `ITensorMPS.jl`.

[2] The integration is recorded by repository commit `1b818160`, "Bring in files from DmrgppPluginSc," dated April 7, 2026.

remain compiled components. Model implementations derive from a common interface and are selected by `ModelSelector`. New models are added as C++ classes, registered with the selector, and compiled into DMRG++.

`DmrgSolver` implements the model-independent growth and finite-sweep stages. It constructs system and environment blocks, obtains target states through Lanczos diagonalization, truncates each block, and transforms states and operators into the retained basis. These retained block bases carry both transformed operators and quantum numbers, allowing the calculation to be divided into symmetry sectors and represented with block-sparse matrices. Lanczos accesses the Hamiltonian through a common matrix-vector interface. DMRG++ can store the sparse Hamiltonian, apply it on-the-fly through the model, or evaluate its Kronecker-product decomposition without constructing the full superblock matrix. The conventional numerical path uses threaded execution, BLAS, and LAPACK. To address the growing heterogeneity of high-performance computing platforms, DMRG++ is being ported to use Kokkos and Kokkos Kernels (Rajamanickam et al., 2021; C. Trott et al., 2021; C. R. Trott et al., 2022). Kokkos is a performance-portability ecosystem that enables a single C++ source code implementation to run efficiently across diverse architectures— from laptop CPUs to GPU-accelerated nodes on leadership-class supercomputers— by abstracting parallel execution patterns and memory layout management. Kokkos Kernels complements this by providing an extensive suite of performance-portable linear algebra primitives, including sparse matrix-vector operations, dense linear solvers, and batched kernels that are critical for DMRG++'s computationally intensive operations. This approach will allow DMRG++ to leverage current and future hardware architectures without maintaining separate code paths for different platforms, while preserving the numerical accuracy and convergence properties that users depend on. For now, an optional integrated batched path can use MAGMA as an accelerator backend.

The targeting layer determines which states contribute to the reduced density matrix during a sweep. Implementations of the `TargetingBase` interface reuse the same model, geometry, basis, and sweep code for ground-state calculations, frequency-dependent response, correction-vector methods, real-time evolution, Chebyshev expansions, ancilla-based finite-temperature evolution, minimally entangled typical thermal states (METTS) (Stoudenmire & White, 2010), and expression-based targeting. The ancilla path reuses the time-step targeting infrastructure for purification, while METTS has a dedicated targeting implementation.

Measurements follow a similar separation. Selected operator expressions can be evaluated in situ during the main run, while the separate `observe` executable reads saved simulation data to calculate one-point and multipoint correlations afterward. The input language allows target vectors to be specified in compact form from products and sums of model operators acting on the ground state or previously defined target states, request time evolution of those vectors, and use the resulting states in braket expressions for measurements. This lets many observables and calculation protocols be specified at runtime that would otherwise require additional development in C++.

DMRG++ is composed of multiple executables that share these layers. The `dmrg` executable runs simulations, `observe` post-processes saved results, `manyOmegas` coordinates calculations at multiple frequencies, and `introspect` examines model bases, operators, and Hamiltonian terms.

The DMRG++ workflow uses HDF5 to store model and solver parameters, the geometry, an encoded copy of the input, energies, target states, renormalized bases, and checkpoint data. These records support restart and recovery and provide saved simulation data for subsequent analysis by tools such as `observe`. Program and source-revision information is also reported with each run.

Project Infrastructure

DMRG++ uses a CMake-based build system to organize its modular architecture. Common implementation code is structured as internal library targets that are linked into multiple command-line executables, promoting code reuse while maintaining clear separation between components. The shared interfaces for models, targeting methods, and matrix-vector operations serve as compiled extension points within the source tree—new physics models or numerical methods can be added by implementing these interfaces and registering them with the appropriate selectors. This design preserves unified solver, input parsing, and testing infrastructure across all extensions. These internal interfaces are not exposed as a stable public library API; instead, they provide structured development pathways for contributors working within the DMRG++ source tree.

Regression tests are based on actual previous scientific applications of DMRG++ using representative input files and compare selected energies and observables with expected results. For new code the Catch2 testing framework has been adopted to support unit testing and tests are being added for existing code when possible.

Research impact statement

The project received research support during two distinct periods between 2011 and 2020 (G. Alvarez, 2011; Elwasif et al., 2017), and a new period of research funding began in 2025, mentioned in the acknowledgments. Except for a gap between 2021 and 2024, continued maintenance has enabled its models, algorithms, numerical kernels, and analysis tools to evolve.

Over time, the scientific scope of DMRG++ expanded beyond ground states to nonequilibrium dynamics, frequency-dependent response, and finite-temperature observables.

As of 2026, the seven authors of this paper maintain DMRG++ as part-time developers. The authors know of approximately 15 active users and collaborators; this figure is not intended as a complete census of everyone who has run the software. More than 80 publications cite the original DMRG++ implementation paper (Gonzalo Alvarez, 2009).[3] This aggregate count provides a conservative measure of the software's visibility in the research literature.

DMRG++ is free and open-source software under the UT-Battelle open-source license, and its development takes place publicly at https://github.com/dmrgpp-project/dmrgpp. The repository exposes the commit history, issues, pull requests, contributor activity, and continuous-integration workflows. Users can therefore inspect changes, report defects, propose new features, and contribute code through the same public development process.

AI usage disclosure

Hermes Agent (Nous Research), using the GPT-5.6-sol model, assisted with planning, drafting, revising, and checking this manuscript. The authors reviewed and edited all AI-assisted material and take responsibility for the accuracy and final content of the paper. AI assistance was also used in selected DMRG++ commits, which identify that assistance in their commit messages.

Acknowledgements

Thanks to all users and contributors to DMRG++, during its history. This work was supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing

[3] Citation records for DOI `10.1016/j.cpc.2009.02.016` were checked in Crossref and OpenAlex on July 16, 2026. Crossref reported 85 citations, while an OpenAlex citing-works query returned 94 records, or 92 after deduplication by DOI. Because database coverage and record handling differ, we report the conservative lower bound "more than 80 publications."

Research and Office of Basic Energy Sciences, Scientific Discovery through Advanced Computing (SciDAC) program under the CONNEQT project.

References

Alvarez, Gonzalo. (2009). The density matrix renormalization group for strongly correlated electron systems: A generic implementation. *Computer Physics Communications*, *180*(9), 1572–1578. https://doi.org/10.1016/j.cpc.2009.02.016

Alvarez, G. (2011). *Diagonalization solvers for electronic collective phenomena in nanoscience (Project ID: 0000201185)*. U.S. Department of Energy, Office of Science, Early Career Research Program; Oak Ridge National Laboratory.

Alvarez, G. (2013). Production of minimally entangled typical thermal states with the krylov-space approach. *Physical Review B*, *87*(24), 245130. https://doi.org/10.1103/PhysRevB.87.245130

Alvarez, G., Dias da Silva, L. G. G. V., Ponce, E., & Dagotto, E. (2011). Time evolution with the density-matrix renormalization-group algorithm: A generic implementation for strongly correlated electronic systems. *Physical Review E*, *84*(5), 056706. https://doi.org/10.1103/PhysRevE.84.056706

Elwasif, W., D'azevedo, E., Alvarez, G., & Hernandez-Mendoza, O. (2017). *Bringing the DMRG++ scientific application to exascale (Project ID: 8423; 2017–2018)*. Oak Ridge National Laboratory, Laboratory Directed Research and Development.

Fishman, M., White, S. R., & Stoudenmire, E. M. (2022). The ITensor software library for tensor network calculations. *SciPost Physics Codebases*. https://doi.org/10.21468/SciPostPhysCodeb.4

Hastings, M. B. (2007). An area law for one-dimensional quantum systems. *Journal of Statistical Mechanics: Theory and Experiment*, *2007*(08), P08024. https://doi.org/10.1088/1742-5468/2007/08/P08024

Motta, M., & Zhang, S. (2018). Ab initio computations of molecular systems by the auxiliary-field quantum monte carlo method. *WIREs Computational Molecular Science*, *8*(5), e1364. https://doi.org/10.1002/wcms.1364

Nocera, A., & Alvarez, G. (2016a). Spectral functions with the density matrix renormalization group: Krylov-space approach for correction vectors. *Physical Review E*, *94*(5), 053308. https://doi.org/10.1103/PhysRevE.94.053308

Nocera, A., & Alvarez, G. (2016b). Symmetry-conserving purification of quantum states within the density matrix renormalization group. *Physical Review B*, *93*(4), 045137. https://doi.org/10.1103/PhysRevB.93.045137

Rajamanickam, S., Acer, S., Berger-Vergiat, L., Dang, V., Ellingwood, N., Harvey, E., Kelley, B., Trott, C. R., Wilke, J., & Yamazaki, I. (2021). *Kokkos kernels: Performance portable sparse/dense linear algebra and graph kernels*. https://arxiv.org/abs/2103.11991

Schollwöck, U. (2011). The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, *326*(1), 96–192. https://doi.org/10.1016/j.aop.2010.09.012

Sehlstedt, P., Brandejs, J., Bientinesi, P., & Karlsson, L. (2026). The software landscape for the density matrix renormalization group. *Computer Physics Communications*, *324*, 110136. https://doi.org/10.1016/j.cpc.2026.110136

Stoudenmire, E. M., & White, S. R. (2010). Minimally entangled typical thermal state algorithms. *New Journal of Physics*, *12*(5), 055026. https://doi.org/10.1088/1367-2630/12/5/055026

Stoudenmire, E. M., & White, S. R. (2012). Studying two-dimensional systems with the density matrix renormalization group. *Annual Review of Condensed Matter Physics*, *3*(1), 111–128. https://doi.org/10.1146/annurev-conmatphys-020911-125018

Trott, C. R., Lebrun-Grandié, D., Arndt, D., Ciesko, J., Dang, V., Ellingwood, N., Gayatri, R., Harvey, E., Hollman, D. S., Ibanez, D., Liber, N., Madsen, J., Miles, J., Poliakoff, D., Powell, A., Rajamanickam, S., Simberg, M., Sunderland, D., Turcksin, B., & Wilke, J. (2022). Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, *33*(4), 805–817. https://doi.org/10.1109/TPDS.2021.3097283

Trott, C., Berger-Vergiat, L., Poliakoff, D., Rajamanickam, S., Lebrun-Grandie, D., Madsen, J., Al Awar, N., Gligoric, M., Shipman, G., & Womeldorff, G. (2021). The kokkos ecosystem: Comprehensive performance portability for high performance computing. *Computing in Science Engineering*, *23*(5), 10–18. https://doi.org/10.1109/MCSE.2021.3098509

White, S. R. (1992). Density matrix formulation for quantum renormalization groups. *Physical Review Letters*, *69*(19), 2863–2866. https://doi.org/10.1103/PhysRevLett.69.2863

White, S. R. (1993). Density-matrix algorithms for quantum renormalization groups. *Physical Review B*, *48*(14), 10345–10356. https://doi.org/10.1103/PhysRevB.48.10345